



Programování v Javě

4IT101

Třída Object a její metody

Třída Object

- Speciální třída
- Každá vytvořená třída v Javě je zároveň typu Object
- Proto všechny třídy mají tyto metody
 - **String toString()**
 - **boolean equals(Object obj)**
 - **int hashCode()**
 - **Class<?> getClass()**
 - **void wait()**
 - **void notify()**
 - **void notifyAll()**
 - **Object clone()**



Metoda toString

- Metoda toString slouží k převedení instance na String
- Automaticky použita
 - Pokud spojujeme String hodnoty pomocí +
 - Pokud použijeme `System.out.println(instance)`
- Pokud tuto metodu nedefinujeme a použijeme toString, dostaneme text s názvem třídy a hexadecimal formátem jeho hashCode
 - `Auto@82f0db`
 - `Clovek@95a5cf`



Metoda toString

- Při definování této metody musíme dodržet tuto hlavičku
`public String toString()`
- Pokud vše definováno správně, metoda se spustí
- Správnost definování můžeme pohlídat anotací `@Override`



Metoda toString - příklad

```
public class Auto {  
  
    private String spz;  
    private int rokVyroby;  
  
    @Override  
    public String toString() {  
        return ("Auto má spz: " + spz + " a rok výroby: " + rokVyroby);  
    }  
}
```



Co jsou anotace?

- Zjednodušeně
 - Slouží k označování metod, tříd atp.
 - Jedná se o speciální druh metadat
 - Většinou přidáváme kvůli dodání extra funkcionality
- Vždy začínají znakem @
- Typické anotace
 - @Override
 - @Deprecated
 - @SuppressWarnings



Anotace Override

- Zpracovávána překladačem Javy
- Přidává kontrolu na správné překrytí metody
 - Například zda metoda toString má stejnou hlavičku jako metoda toString ve třídě Object



Metoda equals

- Slouží k porovnání obsahu dvou instancí
 - Rovnost je typicky vyhodnocována na základě hodnot atributů
- Nespouští se automaticky místo ==
- Bez překrytí porovnává reference v paměti
- Blízká vazba na metodu hashCode
- Hlavička metody

```
public boolean equals(Object o)
```



Metoda equals - pravidla

- Tvorba metody equals má určitá pravidla

```
Auto ford = new Auto("Ford");  
Auto skoda = new Auto("Škoda");  
  
ford.equals(ford); //Musí vratit true  
  
//Musí mít stejný výsledek  
ford.equals(skoda);  
skoda.equals(ford);  
  
skoda.equals(null); //Vrací hodnotu false
```



Metoda equals - postup

- Nejprve porovnáme umístění v paměti (odkazy)
 - Vrátime true, pokud se rovnají
- Porovnáme pomocí instanceof, zda je druhá instance stejného typu
 - Pokud nejsou, vrátíme false
- Provedeme přetypování druhé instance na stejný typ
- Porovnáme všechny rovnost určující atributy



Metoda equals – atributy

- Nemusíme používat gettery (pokud nechceme)
- Porovnávání primitivních datových typů pomocí ==
- Porovnání referenčních typů
 - Pokud datový atribut nemůže být nikdy null
`this.atribut.equals(druhy.atribut)`
 - Pokud datový atribut může být null
`Objects.equals(this.atribut, druhu.atribut)`
- Pokud se jakékoliv porovnání nerovná, vrátíme ihned false



Metoda equals() - příklad

```
public class Auto {  
    private String vyrobce;  
  
    @Override  
    public boolean equals(Object o) {  
        if (this == o) {  
            return true;  
        } else if (!(o instanceof Auto)) {  
            return false;  
        }  
        Auto druhe = (Auto) o;  
        return Objects.equals(this.vyrobce, druhy.vyrobce);  
    }  
}
```



Metoda equals vs ==

```
Auto auto1 = new Auto("Škoda");  
Auto auto2 = auto1;  
Auto auto3 = new Auto("Škoda");  
  
boolean vysledek = auto1 == auto2; //true  
boolean vysledek = auto1 == auto3; //false  
boolean vysledek = auto1.equals(auto2); //true
```



Metoda equals – kdy není potřeba

- Pokud vytváříme třídu tak, že je z podstaty každá instance unikátní
- Pokud porovnávání nepotřebujeme využívat
- Pokud nám vyhovuje implementace předka
 - Více v přednášce o dědičnosti



Metoda hashCode

- Metoda vracející číselnou reprezentaci objektu
 - Snažíme se o co největší unikátnost
 - Konstruováno na základě hodnot datových atributů
- Unikátnost této číselné hodnoty nelze zaručit
- Nepoužívá se pro určení rovnosti objektů
 - Toto číslo pouze naznačuje rovnost
 - Opravdovou rovnost zajišťuje metoda equals
- Hlavička metody

```
public int hashCode()
```



Metoda hashCode

- Používána pro správnou funkci hashových kolekcí
 - Viz HashSet -> později tuto přednášku
- Hashová hodnota typu boolean je 1 pro true a 0 pro false
- Číselné typy a typ char vrací hash hodnotu rovnou své hodnotě přetypované na int



Metoda hashCode - pravidla

- Pro nezměněnou instanci vždy musí vrátit stejné číslo
- Pokud se dvě instance rovnají pomocí metody equals, musí se rovna i jejich hodnoty metod hashCode
 - Nemusí platit obráceně!



Metoda hashCode - postup

- Vybereme stejné datové atributy jako pro metodu equals
- Pro výpočet využijeme metodu „hash“ ze třídy Objects
 - Jednotlivé hodnoty skládáme za sebe
 - Oddělujeme pomocí ,
`Objects.hash(jmeno, prijmeni, rodneCislo);`
- Výsledek metody hash vrátíme



Metoda hashCode - příklad

```
public class Auto {  
    private String vyrobce;  
  
    @Override  
    public int hashCode() {  
        return Objects.hash(vyrobce);  
    }  
  
    //equals  
  
}
```



Datové struktury

Datové struktury

- Kolekce – Collection
 - Seznam – List
 - Množina – Set
 - Fronta – Queue
- Pole – Array
 - Jednorozměrná
 - Vícerozměrná
- Mapa – Map
- Výčtový typ – Enum

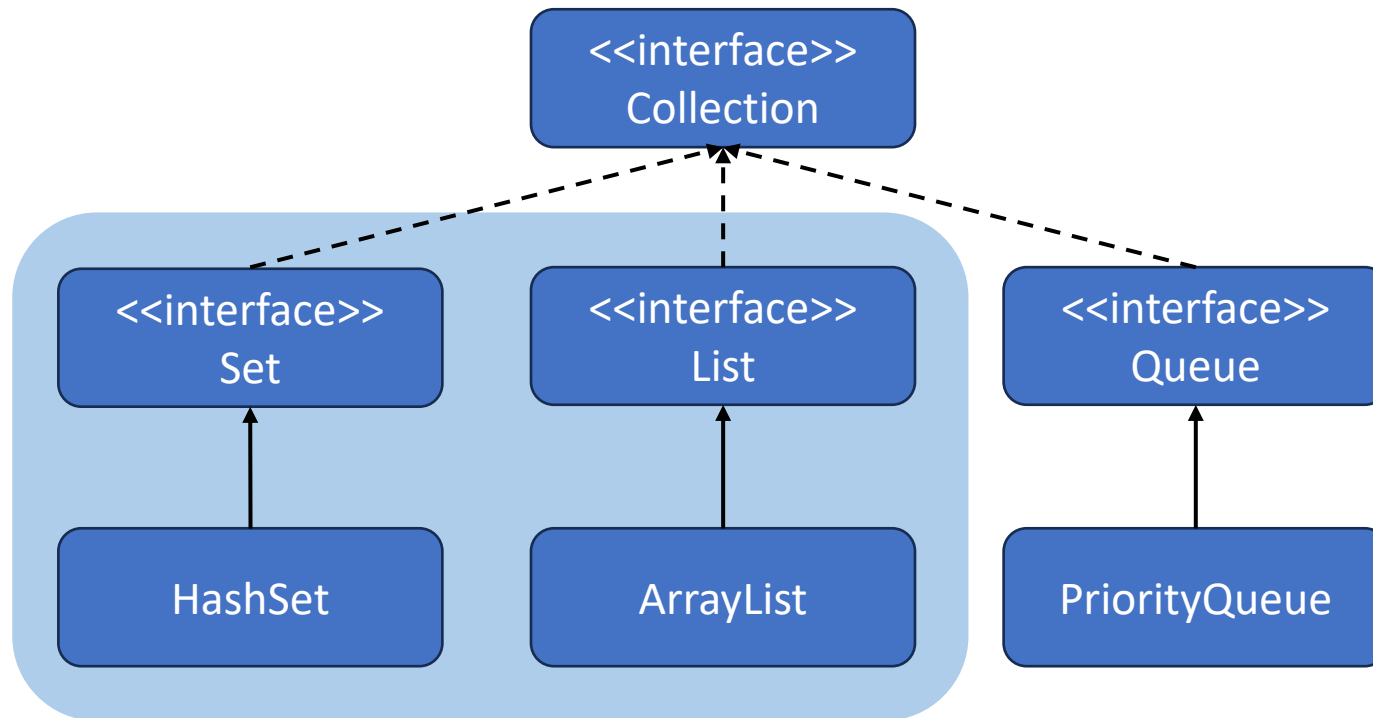


Kolekce - Collection

- Společný interface pro všechny druhy kolekcí
 - Implementace se liší ve způsobu fungování
 - Slouží k ukládání více hodnot v jedné proměnné
 - Nemají maximální kapacitu
 - Dynamicky se rozšiřují
 - Kolekce vyžaduje zadání generického typu
 - Generický typ se vkládá mezi < a >
 - Určuje „co bude ukládáno“ v kolekci
- ```
Collection<Auto> mojeAuto;
```



# Kolekce – Collection



# Kolekce – Společné metody



| Metoda                                    | Popis                                                                                               |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>boolean add(E element)</code>       | Přidá do kolekce prvek typu E. Pokud se operace nepodaří, vrací false.                              |
| <code>void clear()</code>                 | Smaže všechny prvky v kolekci.                                                                      |
| <code>boolean contains(Object o)</code>   | Vrací true, jestliže kolekce obsahuje prvek uvedený jako argument metody.                           |
| <code>boolean isEmpty()</code>            | Vrací true, jestliže je kolekce prázdná.                                                            |
| <code>boolean remove(Object o)</code>     | Jestliže kolekce obsahuje argument, je odpovídající prvek zrušen. Vrací true, když je prvek zrušen. |
| <code>int size()</code>                   | Vrací počet prvků uložených v kolekci.                                                              |
| <code>Iterator&lt;E&gt; iterator()</code> | Vrací instanci rozhraní Iterator, pomocí které je možno procházet tuto kolekci                      |

# Kolekce - List

- Typ List je interface
  - Nejčastěji používaná implementace je nazývána ArrayList
- Umožňují uložit duplicitní záznamy
- Drží pořadí vložení
  - První vložený bude vždy první atp.
- Při přístupu k hodnotám lze používat indexy
  - Index je pořadové číslo položky v listu
  - Vždy začíná od hodnoty 0 a končí  $n-1$  ( $n$  je množství položek v listu)



# Kolekce – List – Vytvoření

```
List<Auto> seznamAut = new ArrayList<Auto>();
List<Auto> seznamAut = new ArrayList<>();
```

```
List<String> seznam = new ArrayList<String>();
List<String> seznam = new ArrayList<>();
```

```
List<Integer> seznamCisel = new ArrayList<Integer>();
List<Integer> seznamCisel = new ArrayList<>();
```



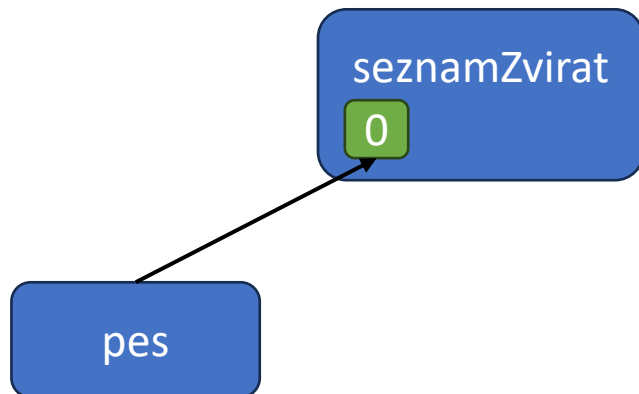
# Kolekce – List metody

| Metoda                                      | Popis                                                                            |
|---------------------------------------------|----------------------------------------------------------------------------------|
| <code>void add(int index, E element)</code> | Přidá do kolekce na zadanou pozici prvek typu E.                                 |
| <code>E get(int index)</code>               | Získání prvku na zadané pozici v seznamu.                                        |
| <code>E remove(int index)</code>            | Zruší prvek ze zadané pozice seznamu, tento prvek vrací jako návratovou hodnotu. |



# Kolekce – List – Přidání

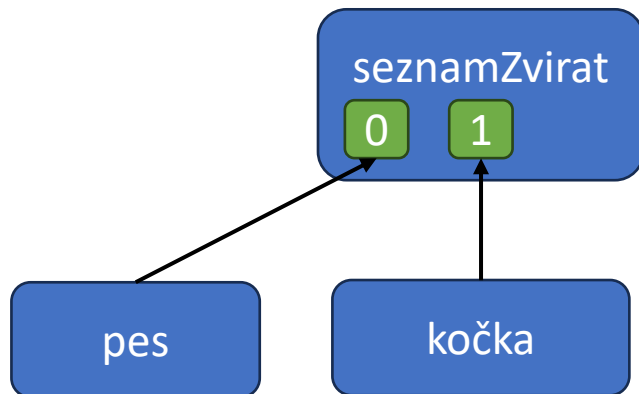
```
List<String> seznamZvirat = new ArrayList<>();
seznamZvirat.add("pes");
```



# Kolekce – List – Přidání

```
List<String> seznamZvirat = new ArrayList<>();

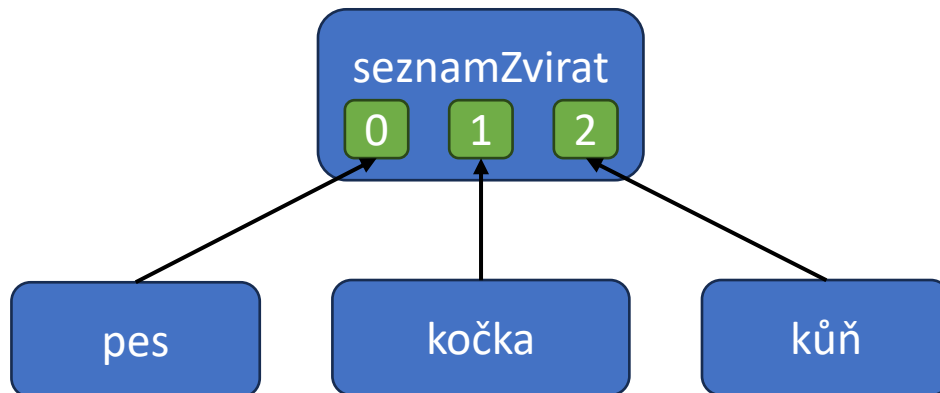
seznamZvirat.add("pes");
seznamZvirat.add("kočka");
```



# Kolekce – List – Přidání

```
List<String> seznamZvirat = new ArrayList<>();

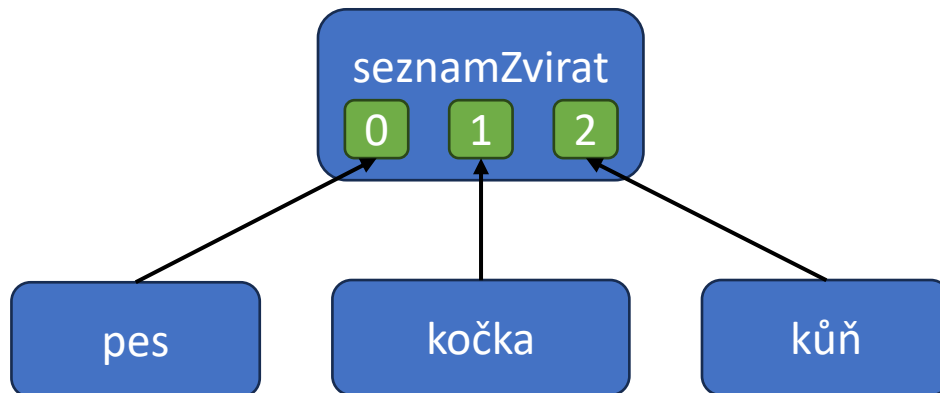
seznamZvirat.add("pes");
seznamZvirat.add("kočka");
seznamZvirat.add("kůň");
```



# Kolekce – List – Přidání na index

```
List<String> seznamZvirat = new ArrayList<>();

seznamZvirat.add("pes");
seznamZvirat.add("kůň");
seznamZvirat.add(1, "kočka");
```

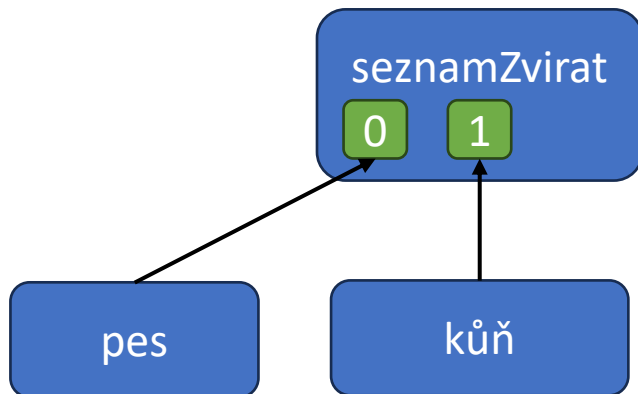


# Kolekce – List - Odstranění

```
List<String> seznamZvirat = new ArrayList<String>();

seznamZvirat.add("pes");
seznamZvirat.add("kočka");
seznamZvirat.add("kůň");

seznamZvirat.remove(1); //Vrácena hodnota "kočka"
```

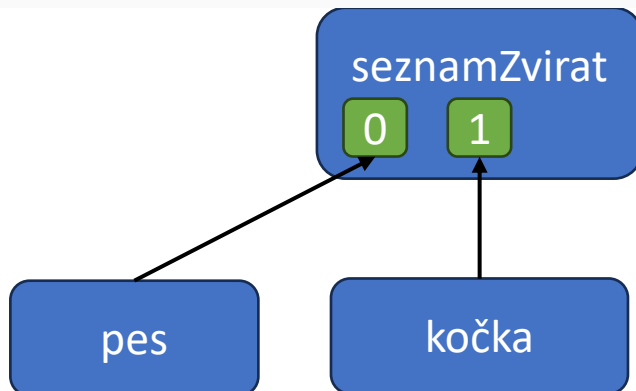


# Kolekce – List - Odstranění

```
List<String> seznamZvirat = new ArrayList<String>();

seznamZvirat.add("pes");
seznamZvirat.add("kočka");
seznamZvirat.add("kůň");

seznamZvirat.remove("kůň"); //Vrácena hodnota true
seznamZvirat.remove("ovce"); //Vrácena hodnota false
```



# Kolekce - Set

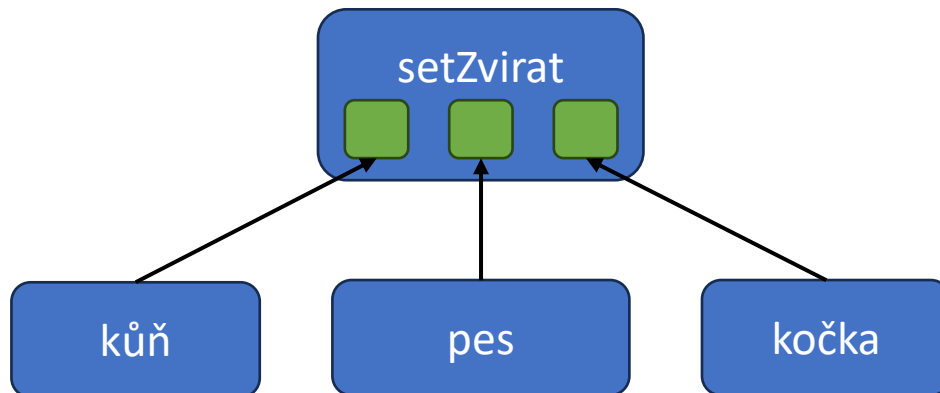
- Typ Set je interface
  - Nejčastěji používaná implementace je nazývána HashSet
- Neumožňuje uložit duplicitní záznamy
  - Využívá hashCode a equals (Implementace HashSet)
  - Pro správnou funkcionality tyto metody musí být vytvořeny VŽDY!
- Nedrží pořadí vložení
  - První vložený může být klidně poslední
- Při přístupu k hodnotám nelze používat indexy



# Kolekce – Set – Příklad

```
Set<String> setZvirat = new HashSet<>();

setZvirat.add("pes");
setZvirat.add("kočka");
setZvirat.add("pes");
setZvirat.add("kůň");
```



# Kolekce – Set – Příklad

```
Set<String> setZvirat = new HashSet<>();
```

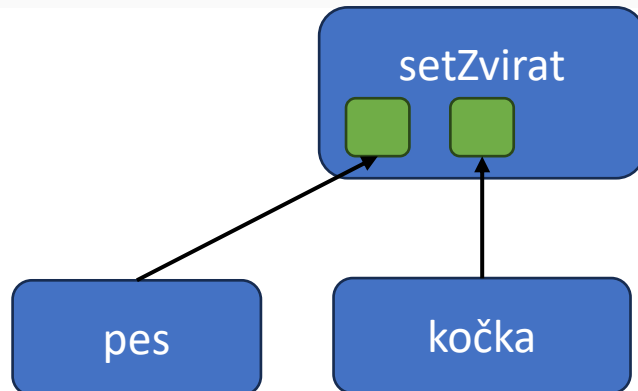
```
setZvirat.add("pes");
```

```
setZvirat.add("kočka");
```

```
setZvirat.add("kůň");
```

```
setZvirat.remove("kůň"); //Vrácena hodnota true
```

```
setZvirat.remove("ovce"); //Vrácena hodnota false
```



# Kolekce – HashSet – Vložení

```
public class Pes {
 private String jmeno;
 private String rasa;

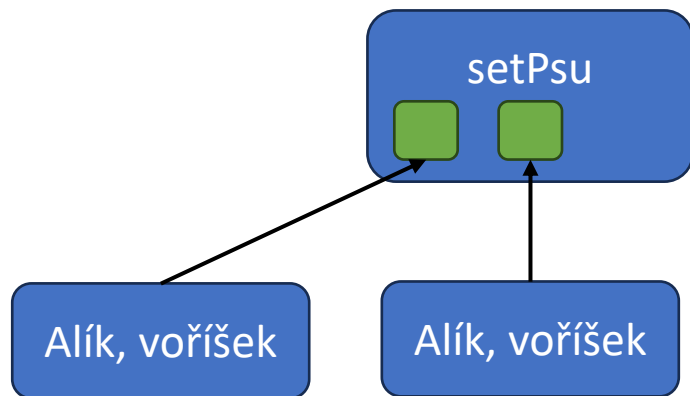
 public Pes(String jmeno, String rasa) {
 this.jmeno = jmeno;
 this.rasa = rasa;
 }
}
```



# Kolekce – HashSet – Vložení

```
Set<Pes> setPsu = new HashSet<>();

setPsu.add(new Pes("Alík", "voříšek"));
setPsu.add(new Pes("Alík", "voříšek"));
```



# Kolekce – HashSet – Vložení

- Pro správné vyfiltrování duplicity musíme přidat metody hashCode a equals
- Nestačí přidat pouze jednu z nich!
- Pokud tyto metody chybí, nedojde ke správnému odstranění duplicit



```
public class Pes {
 ... constructor a datove atributy
 @Override
 public boolean equals(Object o) {
 if (this == o) {
 return true;
 }
 if (o == null || getClass() != o.getClass()) {
 return false;
 }
 Pes pes = (Pes) o;
 return Objects.equals(jmeno, pes.jmeno)
 && Objects.equals(rasa, pes.rasa);
 }

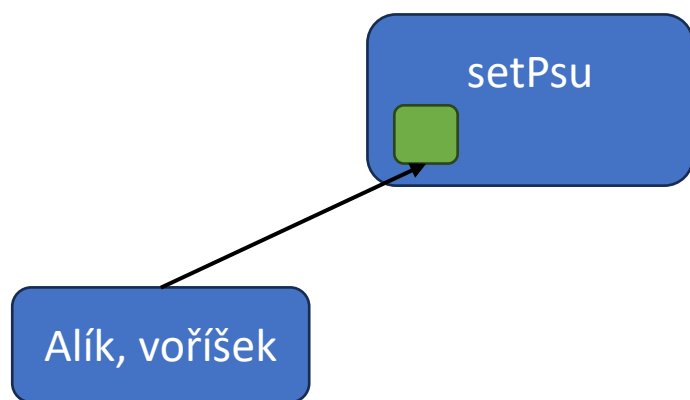
 @Override
 public int hashCode() {
 return Objects.hash(jmeno, rasa);
 }
}
```



# Kolekce – HashSet – Vložení

```
Set<Pes> setPsu = new HashSet<>();

setPsu.add(new Pes("Alík", "voříšek"));
setPsu.add(new Pes("Alík", "voříšek"));
```



# Cyklus - foreach

- „Klasický“ for cyklus – viz minulá hodina

```
for (int i = 0; i < list.size(); i++) {
 System.out.println(seznam.get(i));
}
```

- Cyklus foreach

```
for (String polozka : list) {
 System.out.println (polozka);
}
```

- Můžeme použít s jakoukoliv kolekcí či polem

